Adaptive Momentum Transfer Attacks

for Face Verification (ADAMSI_FGM)

Bhumika Singh

July 10, 2026

Attacks Considered

- PGD (baseline)
- MI-FGSM (momentum-based baseline)
- TI-FGSM (baseline)
- SI-NI-FGSM (baseline)
- MI-ADMIX-DI-TI (baseline)

New attack added this assignment:

- ADAMSI_FGM — adaptive momentum & adaptive step size (AAAI 2024, adapted for face verification)

ADAMSI-FGM (Adaptive Momentum Iterative FGSM)

Definition & Nature

- Transfer-based (black-box) adversarial attack
- Gradient-based, iterative optimization
- L∞-bounded constrained attack
- Extension of MI-FGSM

Basic Idea

- Replaces MI-FGSM's fixed momentum decay with a time-decaying adaptive coefficient
- Replaces sign()-based updates with an adaptive per-pixel step size

Difference from MI-FGSM

- Momentum coefficient $\beta_1/\sqrt{t}$ decays over iterations
- Step scaled by accumulated squared gradients, not sign()

Working of ADAMSI-FGM

1. Initialize adversarial image as original input
2. Initialize momentum $m_0 = 0$, accumulator $v_0 = 0$
3. Define attack objective (cosine similarity)
4. For T iterations: compute gradient, update adaptive momentum & step, apply update
5. Project into ε-bounded region, clip to valid pixel range
6. Return final adversarial image

Attack Implementation

```
def adamsi_fgm(model, x, tgt_emb,
    attack_type, beta1=0.9,
    eps_adapt=1e-8):

  adv = tf.identity(x)
  m = tf.zeros_like(x)
  v = tf.zeros_like(x)
  tgt_emb = tf.nn.l2_normalize(
      tgt_emb, axis=1)
  alpha0 = EPSILON / NUM_ITER

  for t in range(1, NUM_ITER+1):
    with tf.GradientTape() as tape:
      tape.watch(adv)
      emb = compute_embedding(model, adv)
      cos = tf.reduce_sum(
          emb * tgt_emb, axis=1)
      loss = attack_loss(cos,
          attack_type)
    grad = tape.gradient(loss, adv)
    grad_n = grad / (tf.reduce_mean(
        tf.abs(grad)) + 1e-8)

    beta1_t = beta1 / tf.sqrt(
        tf.cast(t, tf.float32))
    m = beta1_t*m + (1-beta1_t)*grad_n

    v = v + tf.square(grad_n)
    step = m / (tf.sqrt(v)+eps_adapt)

    adv = adv + alpha0*NUM_ITER*step
    adv = tf.clip_by_value(
        adv, x-EPSILON, x+EPSILON)
    adv = tf.clip_by_value(adv,-1.,1.)
  return adv
```

ADAMSI_FGM in Our Implementation

1. Time-decaying momentum coefficient $\beta_1/\sqrt{t}$ (vs. fixed DECAY in MI-FGSM)
2. Second-moment accumulator v tracks squared gradients
3. Adaptive step = m / ($\sqrt{v}$ + ε), replacing sign()
4. Same ε-ball clipping strategy as MI-FGSM/PGD
5. Written in TensorFlow/Keras (GradientTape), matching existing baseline style
6. Integrated via ALL_ATTACKS, ATTACK_COLS, adamsi_fgm(), and run_attack()

Results vs. Baseline

Attack	Rows Tested	Breach Rate	Mean Impact
ADAMSI_FGM (new)	**90**	**37.78%**	**0.140**
SI_NI_FGSM (baseline)	480	29.17%	0.176
MI_FGSM (baseline)	480	26.67%	0.164
MI_ADMIX_DI_TI (baseline)	480	24.17%	0.153
TI_FGSM (baseline)	480	20.42%	0.131
PGD (baseline)	480	16.67%	0.100

ADAMSI_FGM achieved the highest breach rate among all attacks evaluated on the subset.

Attack Goal	Breach Rate	Mean Impact
Dodging (evade correct match)	44.44%	0.178
Impersonation (force false match)	31.11%	0.101

Interpretation & Limitations

Interpretation

- Adaptive momentum + adaptive step improved cross-model transferability vs. all evaluated baselines
- Stronger at dodging (44.4%) than impersonation (31.1%) — an expected pattern, since pushing an embedding away from a fixed target is an easier optimization target than pushing it precisely toward one

Limitations & Next Steps

- Only Facenet512 tested as surrogate; other surrogates (ArcFace, GhostFaceNet, VGG-Face) would give fuller coverage
- IR152 victim evaluation skipped — requires a separate PyTorch loading pipeline
- This is a simplified adaptation of the paper's convergence-driven update rule, not a line-for-line reproduction of the authors' implementation

Thank You